

The data used for the code for this assignment can be found here:

https://drive.google.com/drive/folders/1eY0sYrnXD7MjEaHPZhM37JURR759o9cP?usp=drive_link

1. Question 1:

a. The required network statistics are given as follows:

1. Number of Nodes	1458 nodes
2. Number of Edges	1948 edges
3. Density of the graph	0.0018340107310340413
4. Diameter of the graph	19

Table 1: Required Network Statistics

b. The required average centrality values are given as follows:

1. Degree Centrality	0.0018340107310340413
2. Average Betweenness Centrality	0.003992009625731886
3. Average Closeness Centrality	0.1509653441907969

Table 2: Required Average Centrality Measures

1. Degree Centrality

- In the networkx package, degree centrality of a node is defined as the degree possessed by the node in the network divided by the number of nodes in the network minus one. We report the average degree centrality of the network, which is given as the sum of the degree centralities of all nodes divided by the total number of nodes in the graph.
- The degree centrality of a node, in a protein-protein interaction network (PPIN), denotes the number of proteins with which a given protein can interact, typically with a certain level of confidence, which may be represented using edge weights. This

interaction could be a physical interaction or a functional association, either experimentally verified or computationally predicted.

- A protein whose node has a relatively high degree centrality, thus, interacts with relatively more proteins in the network than others, and may be involved in multiple signalling pathways or may regulate multiple other proteins.

2. Betweenness Centrality

- Betweenness centrality of a node is given as the sum of the fractions of ‘all-pairs’ shortest paths that do not commence or terminate at the node but pass through it. We report the average betweenness centrality of the network, which is given as the sum of the betweenness centralities of all nodes divided by the total number of nodes in the graph.
- Betweenness centrality denotes how likely a given node is situated on paths that connect two nodes in the graph. Thus, in a PPIN, betweenness centrality denotes how often a given protein is an intermediate involved in the series of reactions that take a substrate from a reaction catalysed by one protein to that catalysed by some other protein.
- Thus, a protein whose node has a high betweenness centrality is a protein that lies at the intersection of multiple cascades of reactions that connect different proteins or reactions. This protein, for example, could lie at an important juncture in multiple signalling pathways, through which multiple signalling cascades must pass (for example, a specific kinase enzyme); such a protein can serve as a connecting link between different pathways.

3. Closeness Centrality

- Closeness centrality of a node is given as the reciprocal of the average of the shortest path lengths from the node to all the other nodes in the network. Thus, closeness centrality denotes how ‘central’ a particular node is in the network. We report the average betweenness centrality of the network, which is given as the sum of the betweenness centralities of all nodes divided by the total number of nodes in the graph.
- The closeness centrality of a node denotes the fraction of nodes in the network with which shortest paths from the given node exist; thus, in a yeast PPIN, these nodes represent the proteins that are the ones likely to get affected, directly or indirectly (depending on the lengths of the shortest paths connecting them), by the function of the protein represented by the given node.
- For example, the knock-out of a gene encoding a relatively more ‘central’ protein is likely to affect the functions of multiple other proteins to which it is connected by ‘shortest paths’, as compared to the knock-out of a gene encoding a relatively

less ‘central’ protein, which may not affect the functions of an equally large number of proteins in the network (which has longer shortest paths to the rest of the proteins, and, thus, has lesser closeness centrality).

- c. The top 5% of the nodes (72 nodes) that have the highest respective centralities in the network are given as follows:

1. Top 5% nodes with the highest degree centrality	147, 819, 98, 567, 844, 638, 27, 1275, 961, 1240, 595, 136, 367, 72, 719, 1130, 345, 433, 134, 257, 854, 122, 223, 252, 355, 633, 755, 1315, 81, 154, 267, 563, 979, 1212, 12, 25, 131, 271, 301, 513, 760, 920, 1101, 1113, 1332, 281, 406, 458, 942, 970, 1067, 11, 85, 320, 333, 341, 465, 524, 603, 811, 1041, 1368, 241, 348, 436, 448, 791, 974, 1164, 1294, 20, 43
2. Top 5% nodes with the highest betweenness centrality	819, 844, 98, 147, 595, 1275, 638, 567, 1240, 253, 27, 961, 465, 90, 664, 302, 1101, 241, 122, 367, 384, 979, 1, 1130, 337, 1131, 136, 854, 17, 920, 719, 1164, 433, 1168, 72, 355, 266, 563, 257, 267, 1294, 585, 445, 1212, 603, 535, 584, 345, 1392, 689, 252, 281, 1110, 464, 1014, 942, 1132, 1041, 154, 1113, 458, 1368, 1273, 970, 572, 998, 802, 204, 966, 322, 513, 11
3. Top 5% nodes with the highest closeness centrality	819, 844, 98, 17, 302, 90, 179, 966, 1131, 567, 731, 355, 433, 664, 1240, 253, 252, 609, 513, 585, 241, 13, 337, 204, 551, 1172, 464, 1171, 595, 465, 937, 584, 1266, 180, 1273, 1077, 638, 324, 979, 445, 603, 122, 854, 1275, 563, 1132, 384, 1130, 261, 1368, 267, 662, 681, 920, 1115, 152, 572, 791, 494, 27, 85, 8, 332, 579, 604, 1035, 1212, 524, 184, 435, 831, 526

Table 3: Top 5% nodes with respect to different centrality measures

- d. Upon removal of each of these node sets representing putative essential proteins, the sizes of the largest connected components are given as follows:

1. Size of the largest connected components upon removal of the top 5% nodes with the highest degree centrality	621 nodes
2. Size of the largest connected components upon removal of the top 5% nodes with the highest betweenness centrality	681 nodes
3. Size of the largest connected components upon removal of the top 5% nodes with the highest closeness centrality	1117 nodes

Table 4: The sizes of the largest connected components upon targeted node removal

- e. The distribution of the size of the largest connected component upon a realisation of the random removal of 5% of the total nodes gave the following parameters:

1. Mean	1321.13
2. Standard Deviation	25.283455064527868

Table 5: Measures of Central Tendency for the ensemble of random networks

The Z-scores for the size of the largest connected component upon the execution of each of the targeted node removal strategies, and the associated p-values are given as follows:

Null Hypothesis: The size of the largest connected component obtained on targeted removal does not differ from the mean size of the largest connected component obtained upon random removal.

Alternative Hypothesis: The size of the largest connected component obtained on targeted removal differs from the mean size of the largest connected component obtained upon random removal.

Strategy for targeted removal	Absolute value of Z-score	p-value
1. Degree Centrality-based removal	27.69123121081134	8.904267789385343e-169
2. Betweenness Centrality-based removal	25.318137824370705	2.0169374740455914e-141
3. Closeness Centrality-based removal	8.073659216235443	6.822211606551863e-16

Table 6: Z-scores and p-values for different targeted removal strategies

- A relatively low p-value (typically, lower than the threshold 0.05 value) indicates that the probability of obtaining the result based on random chance is very low, thus suggesting that the result differs from the mean of the distribution due to the inherent nature of the strategy of targeted removal, and not due to random chance.
- Hence, we reject the null hypothesis in each of these three cases. Therefore, the size of the largest connected component obtained on targeted removal differs from the mean size of the largest connected component obtained upon random removal in a statistically significant manner.
- A lower p-value is indicative of a more statistically significant result, and here, also indicates a higher Z-score and a smaller size of the largest connected component. Therefore, based on these metrics, we conclude that degree centrality best supports the centrality-lethality hypothesis for the given network, which suggests that nodes with a higher centrality value are more likely to be essential for cellular function than those with a lower centrality value.
- It is closely followed by betweenness centrality (which generates approximately the same size of the largest connected component), while closeness centrality comes a distant third (with a substantially larger largest connected component upon node removal). These results are reflected in the respective Z-scores and p-values as well.
- Here, gene or protein essentiality is measured based on the size of the largest connected component achieved after node set removal, i.e., the smaller the size of the largest connected component, the more 'lethal' is the specific strategy of targeted removal.
- Suitability of the use of the Z-test:
 - A Z-score is used to perform a hypothesis test when the population is known to possess a normal distribution, and the population variance and mean are known.

- Here, since we obtain standard deviation (or variance) and mean from 100 iterations of random node removal, these measures must be considered as ‘sample’ mean and standard deviation, and not ‘population mean and variance.
- Thus, a two-tailed Student’s t-test is more appropriate for this purpose, since it allows the use of ‘sample’ mean and standard deviation in its calculations. However, this test also implies an underlying normal distribution.
- To determine whether the frequency of the size of the largest connected component followed a normal distribution, the following histogram was plotted:

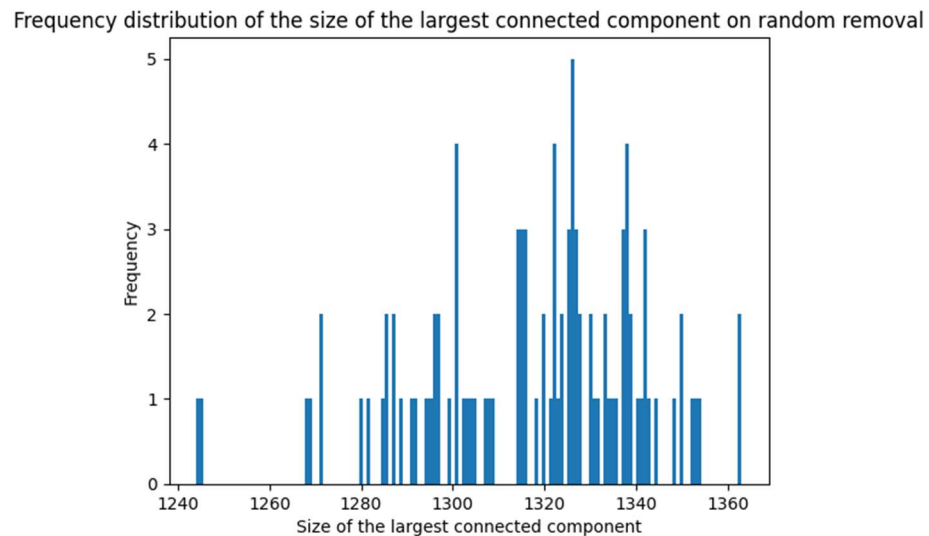


Figure 1: Frequency distribution of the size of the largest connected component on random removal of nodes.

- Therefore, from Figure 1, it is evident that a normal distribution is not followed, as indicated by a left tail indicative of a left-skewed distribution. This can be verified by performing more iterations before plotting the histogram, or by performing other statistical tests (such as MLE and goodness of fit), to determine whether a normal distribution is indeed observed in this case.
 - However, as primary results, we report the above conclusion. Furthermore, the substantially low p-values indicate that targeted removal performs statistically significantly better than random removal in terms of being lethal to the PPIN.
- f. The following steps were used to determine nodes with ‘relatively high Betweenness Centrality but relatively low Degree Centrality’:
1. Consider the top 50 nodes with the highest Betweenness Centrality.
 2. For each of these nodes, determine their Degree Centrality.

3. If the Degree Centrality is less than 10% of the maximum Degree Centrality of a node in the network, report this node as one that satisfies the above criteria.

The following nodes were observed to follow these criteria:

The nodes with relatively high Betweenness Centrality but relatively low Degree Centrality are given as follows:

Node 1 with degree centrality 0.0034317089910775563 and betweenness centrality 0.035417439052021715.
Node 17 with degree centrality 0.0034317089910775563 and betweenness centrality 0.03282562701462217.
Node 266 with degree centrality 0.002059025394646534 and betweenness centrality 0.03099307615665204.
Node 535 with degree centrality 0.002059025394646534 and betweenness centrality 0.026942956981392707.
Node 1392 with degree centrality 0.0034317089910775563 and betweenness centrality 0.02611982261656477.

The node 147 has the maximum degree centrality 0.038435140700068635 and the node 819 has the maximum betweenness centrality 0.21299432211070649 in the graph.

Figure 2: Nodes with ‘relatively high Betweenness Centrality but relatively low Degree Centrality’

These nodes may represent proteins that interact with a relatively low number of proteins, but are more ‘between’ different communities or modules of nodes. Thus, these proteins may potentially serve as links between different clusters of proteins that have similar functions or participate in specific pathways. Thus, the presence of such nodes may be important for connecting different cellular processes (for instance, the output of one could be an input for the other). These proteins could also represent regulatory proteins that do not have multiple interacting partners, but are responsible for regulating key processes such as transcription, translation, post-translational modifications of certain other proteins that are more ‘central’, etc. Thus, the proteins identified above can serve as essential proteins in the PPIN.

- g. The top 2 nodes with the highest closeness centrality in the given edge-list representation are as follows:

1. YDL160C (Closeness Centrality = 0.6851832405754489)

- The label YDL160C refers to the DHH1 (DEAD-box Helicase Homolog 1) gene, whose protein product regulates protein turnover from mRNA in the cytoplasm. This ATP-dependent RNA helicase promotes decapping of the 5’-cap and deadenylation of the 3’-tail in coordination with decapping and deadenylase complexes, thereby leading to mRNA decay in the cytoplasm.
- It is also known to activate the translation of certain mRNAs to promote specific cellular phenotypes such as filamentous growth, mating, and autophagy in *Saccharomyces cerevisiae*.
- Thus, the YDL160C is likely to be connected by several shortest paths of relatively small length to multiple other proteins in the network due to its ability to regulate translation of mRNA in the cytoplasm by acting on key general structural features of mRNA molecules, which could be the reason behind its more ‘central’ position, reflected by its high closeness centrality in the network. Thus, based on the function of the gene DHH1, it can be concluded that this result can be justified based on the biological function of the protein.

2. YAL021C (Closeness Centrality = 0.6469363474122546)

- The YAL021C label represents the CCR4 (Carbon Catabolite Repression 4) protein, which is a part of the CCR4-NOT complex that regulates both transcription and translation (by acting as an mRNA denuclease that deadenylates the molecule by shortening the poly-A 3' tail).
 - Thus, the YAL021C is likely to be connected by shortest paths of relatively small length to multiple other proteins in the network due to its ability to regulate the primary steps involved in the transfer of information in the cell, which may be reflected in its high closeness centrality in the network.
 - Thus, based on the function of the gene CCR4, it can be concluded that this result can be justified based on the biological function of the protein. It would also be interesting to investigate whether the YAL021C node has a relatively high degree centrality since it is known to regulate translation as well as transcription.
 - Thus, the biological function of proteins represented by the top 2 nodes with the highest closeness centrality reflects their potential to be 'connected' by shortest paths of relatively small length not only to other regulatory proteins of certain complexes (physical interaction), but also those whose translation they may repress (functional association), thereby justifying their high closeness centrality values in the network.
-

2. Question 2:

The real-life split in Zachary's Karate Club graph is given as follows:

Community	Number of 'Mr. Hi' nodes	Number of 'Officer' nodes
0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 16, 17, 19, 21	17	0
9, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33	0	17

Table 7: Communities in Zachary's Karate Club graph

The modularity of Zachary's Karate Club graph is 0.39143756676224206.

- a. Upon implementing the greedy modularity optimization algorithm, the following results were obtained:

Maximum number of communities	Modularity	Community sizes	Communities	Number of 'Mr. Hi' nodes	Number of 'Officer' nodes
2	0.3998332115215232	[17, 17]	[0, 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 16, 17, 19, 21], [8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33]	[16], [1]	[1], [16]
3	0.4109649369389629	[17, 9, 8]	[8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33], [1, 2, 3, 7, 9, 12, 13, 17, 21], [0, 16, 19, 4, 5, 6, 10, 11]	[1], [8], [8]	[16], [1], [0]
4	0.4109649369389629	[17, 9, 8]	[8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33], [1, 2, 3, 7, 9, 12, 13, 17, 21], [0, 16, 19, 4, 5, 6, 10, 11]	[1], [8], [8]	[16], [1], [0]

Table 8: Communities obtained from the greedy modularity optimization algorithm

- When the maximum number of communities (max_c) is constrained, modularity initially increases from max_c = 2 to max_c = 3; however, there is no change in the community structure when max_c increases to 4 from max_c = 3. This is the partition with the highest modularity.
- Since the maximum number of communities is constrained, when max_c = 2, the algorithm does not provide a community structure with greater modularity, and the number of communities is restricted to 2. When max_c = 3, an optimum community structure is achieved; this is retained even when max_c = 4, since the max_c constraint is not violated, and a maximum modularity is achieved.
- However, this community structure with maximum modularity does not match the real split of Zachary's Karate Graph. These two partitions are not equal in terms of both the number of communities and modularity. The community structure obtained from the greedy modularity optimization algorithm has both higher modularity and more communities than Zachary's Karate Club graph.

b. The first 5 successive partitions produced by the Girvan-Newman Algorithm are as follows:

Iteration	Number of communities	Modularity	Community sizes	Communities	Number of 'Mr. Hi' nodes	Number of 'Officer' nodes
1	2	0.3476602762317048	[15, 9]	({0, 1, 3, 4, 5, 6, 7, 10, 11, 12, 13, 16, 17, 19, 21}, {2, 8, 9, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33})	[15], [0]	[2], [17]
2	3	0.3423192968647514	[15, 18, 1]	({0, 1, 3, 4, 5, 6, 7, 10, 11, 12, 13, 16, 17, 19, 21}, {32, 33, 2, 8, 14, 15,	[15], [2], [0]	[0], [16], [1]

				18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31}, {9})		
3	4	0.3580611307884035	[10, 18, 5, 1]	({0, 1, 3, 7, 11, 12, 13, 17, 19, 21}, {32, 33, 2, 8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31}, {4, 5, 6, 10, 16}, {9})	[10], [2], [5], [0]	[0], [16], [0], [1]
4	5	0.3849721706864564	[10, 6, 5, 12, 1]	({0, 1, 3, 7, 11, 12, 13, 17, 19, 21}, {2, 24, 25, 27, 28, 31}, {4, 5, 6, 10, 16}, {32, 33, 8, 14, 15, 18, 20, 22, 23, 26, 29, 30}, {9})	[10], [1], [5], [1], [0]	[0], [5], [0], [11], [1]
5	6	0.3757800640917524	[9, 6, 5, 12, 1, 1]	({0, 1, 3, 7, 12, 13, 17, 19, 21}, {2, 24, 25, 27, 28, 31}, {4, 5, 6, 10, 16}, {32, 33, 8, 14, 15, 18, 20, 22, 23,	[9], [1], [5], [1], [0], [1]	[0], [5], [0], [11], [1], [0]

				26, 29, 30}, {9}, {11})		
--	--	--	--	----------------------------	--	--

Table 9: Communities obtained from the Girvan-Newman Algorithm

- When the Girvan-Newman Algorithm is applied, modularity initially decreases from the 1st iteration (2 communities) to the 2nd iteration (3 communities), after which it increases with every iteration until the 4th iteration (5 communities), following which modularity decreases after the 5th iteration (6 communities).
- The partition obtained after the 4th iteration has the highest modularity.
- However, this community structure with maximum modularity does not match the real split of Zachary's Karate Graph. These two partitions are not equal in terms of both the number of communities and modularity. The community structure obtained from the Girvan-Newman algorithm has lower modularity but more communities than the Zachary's Karate Club graph.

c. The two algorithms are compared as follows:

1. Maximum modularity achieved:

- The greedy modularity optimization algorithm follows a bottom-up or agglomerative approach, whereby clusters (initially, all nodes are clusters unto themselves) are merged in an attempt to maximize modularity. Thus, this algorithm can produce community structures with relatively higher values of modularity.
- The community structure obtained from the Girvan-Newman algorithm has lower modularity, while that obtained from the greedy modularity optimization algorithm has a higher modularity than Zachary's Karate Club graph.
- The Girvan-Newman (GN) algorithm starts with the original graph, and removes edges based on edge-betweenness at every step; thus, it follows a divisive clustering approach; thus, since modularity is not optimized at every step, and various nodes or clusters are not merged, as in the case above, the GN algorithm, in some cases, generates community structures with lower modularity than the greedy modularity optimization algorithm.
- This was observed in our example as well.

2. Time complexity:

- The time complexity of the greedy modularity optimization algorithm is $O(md \log n)$, for a graph with n nodes and m edges, where d is the depth of the dendrogram used to represent the communities in the graph.
- For sparse, real-world hierarchical networks, m and d can be approximated as n and $\log n$, respectively; effectively, the time complexity is given by $O(n(\log n)^2)$.
- The time complexity of calculating the edge-betweenness values for all edges in a graph with n nodes and m edges is $O(mn)$; in the Girvan-Newman (GN) Algorithm, this calculation is performed m times; thus, the time complexity of the algorithm is $O(m^2 n)$. However, the running time complexity may be better than the worst-case time complexity, since not all edge-betweenness values may change during every iteration of the algorithm.

3. Ability to detect real-life community structure:

- The GN algorithm is preferred for community detection in smaller graphs; its time complexity may substantially increase for large or dense graphs where $m \sim n$. The greedy modularity optimization algorithm is preferred for large, real-life networks.
- While the GN algorithm is better at detecting smaller clusters, the greedy modularity optimization algorithm suffers from a ‘resolution limit’; it is known to merge communities to increase modularity, thereby incorrectly merging sets of nodes that form distinct clusters in the real-world network.
- Thus, the structure of the real-world graph and the clusters therein determine which algorithm may be better in detecting the real-life community structure.
- In our case, as discussed above, neither of the two algorithms could recover the real-life split of Zachary’s Karate Club graph in the community structure with maximum modularity.
- We also take a look at the 2-community structure produced by the two algorithms:

Number of communities	Modularity	Community sizes	Communities	Number of ‘Mr. Hi’ nodes	Number of ‘Officer’ nodes
2 (GN algorithm)	0.3476602762317048	[15, 9]	({0, 1, 3, 4, 5, 6, 7, 10, 11, 12, 13, 16, 17, 19, 21}, {2, 8,	[15], [0]	[2], [17]

			9, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33})		
2 (Greedy Modularity Optimization Algorithm)	0.3998332115215232	[17, 17]	[0, 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 16, 17, 19, 21], [8, 14, 15, 18, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33]	[16], [1]	[1], [16]

Table 10: 2-community structures obtained from the two algorithms implemented above

- While neither of the 2-community structures obtained from the two algorithms faithfully recovers the real-life split, they come close to doing so.
 - The modularity values obtained in these two cases are comparable to the one obtained from Zachary's Karate Club graph.
- d. NetworkX implements the following function to determine the edge with the maximum betweenness. This has been obtained from the source code for the `nx.girvan_newman()` function in the NetworkX package.

```

if most_valuable_edge is None:

    def most_valuable_edge(G):
        """Returns the edge with the highest betweenness centrality
        in the graph `G`."""

        # We have guaranteed that the graph is non-empty, so this
        # dictionary will never be empty.
        betweenness = nx.edge_betweenness centrality(G)
        return max(betweenness, key=betweenness.get)

```

Figure 3: Identifying the edge with maximum betweenness centrality in the NetworkX implementation of the GN algorithm

NetworkX removes the edge returned by the function shown above at every step of the GN algorithm. Thus, in case of multiple edges with the same betweenness, the edge that appears first in the sorted list formed upon implementing the `max()` function, as shown above.

The other edges will get removed in the subsequent steps based on whether they have the maximum betweenness at any step (note that after computing edge-betweenness again for the next step, some edges may have a different betweenness value than in the previous step due to the removal of an edge before the re-calculation of edge-betweenness values).

e. Resolution limit of the greedy modularity optimization algorithm:

- In the implementation of the algorithms to solve the above problems, no cases of the algorithm merging smaller communities to boost the modularity score were observed.
 - When the algorithm was instead run by constraining the minimum number of communities using the ‘cutoff’ attribute, it initially appeared that the algorithm did merge communities in an attempt to increase modularity.
 - However, it was later realized, upon considering the attributes of the nodes being merged at each of the steps, that the merger of nodes with larger communities not only helped increase modularity, but also classified nodes correctly into groups of nodes that have a preponderance of nodes with either of the two labels, Mr. Hi and Officer, from Zachary’s Karate Club graph.
-

3. Question 3:

- The two networks were loaded in Cytoscape as follows:
 1. Within the Import option in the File tab, select the ‘Table from File’ option.
 2. Upon importing the ‘q3-edges.xlsx’ file from its location, select the ‘Hospital’ or ‘Metro’ sheet, and determine which columns act as Source Node, Target Nodes, or Edge attributes.
 3. The ‘Metro’ network is loaded in the same way. A node that forms multiple self-loops with itself is removed using the ‘Cut’ or ‘Remove Node’ option, so that it does not interfere with further analyses.
- a. To obtain the degree distribution of the networks, the following steps were followed:

1. Select a particular network, and run the Network Analyzer option available in the Tools tab. Let the Analyzer analyse the network as an undirected graph.
2. The CyPlot application within Cytoscape was downloaded for the purpose of plotting the degree distribution. The CyPlot option to plot different graphs is available in the Tools tab; select the Histogram Plot therein and select 'Degree' as the column to plot the Histogram.
3. The degree distribution can then be observed in a CyBrowser window, where aspects such as bin size and bin width of the histogram can be altered in the Traces window within the Styles tab (here, bin size has been set equal to 1).
4. The degree distributions for the two microbial association networks are as follows:

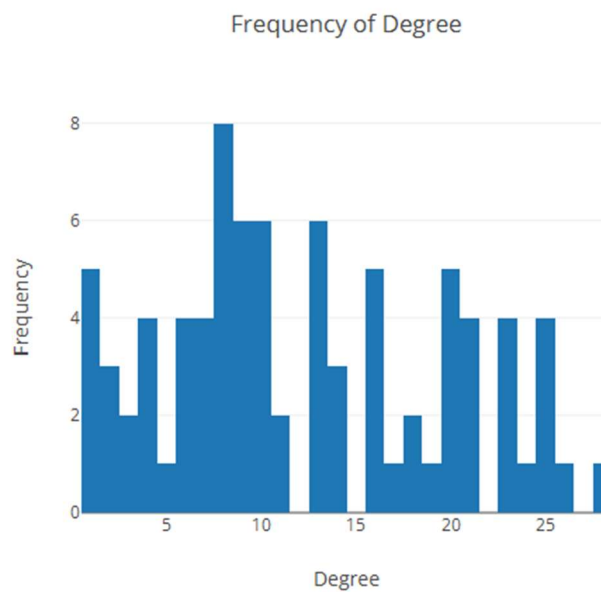


Figure 4: Degree Distribution for the Hospital Network

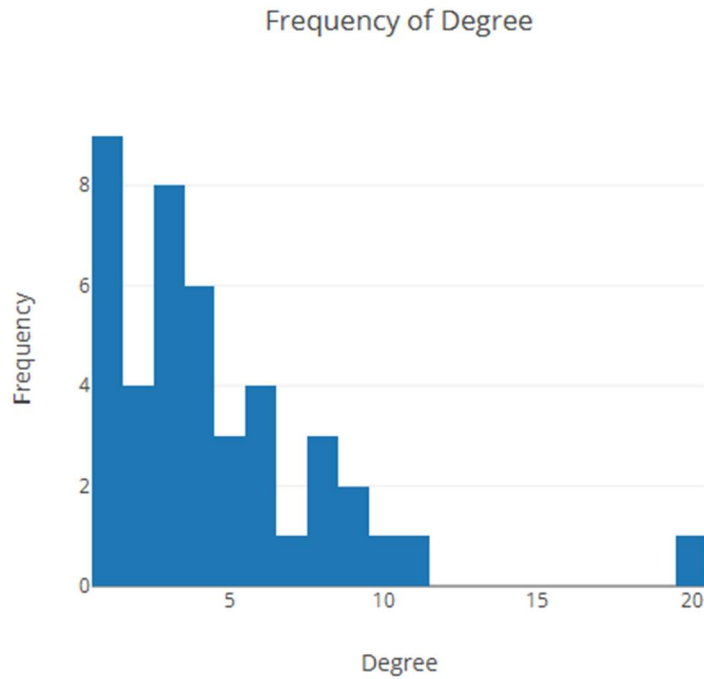


Figure 5: Degree Distribution for the Metro Network

- b. Since Cytoscape does not readily provide methods to compute the average clustering coefficient, the Node Tables for both the graphs were imported in .csv format to a Python Notebook, where a script was written to calculate the average clustering coefficients of all nodes that have the same degree, and then obtain the desired plot for each of the two microbial association networks. The plots (shown as scatter plots) are as follows:

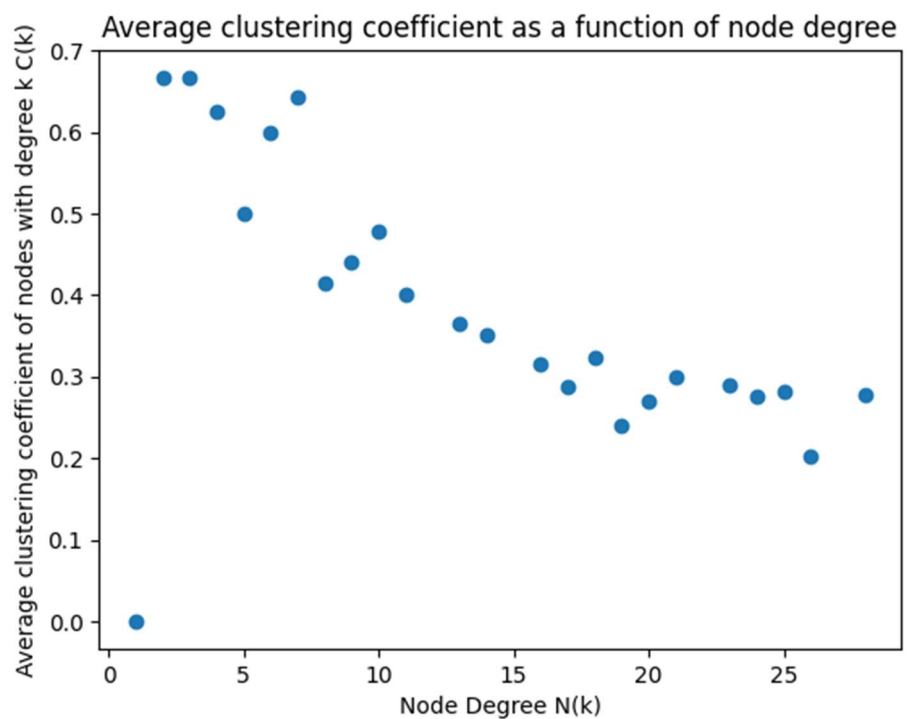


Figure 6: The average clustering coefficient versus node degree plot for the microbial association network in the Hospital environment

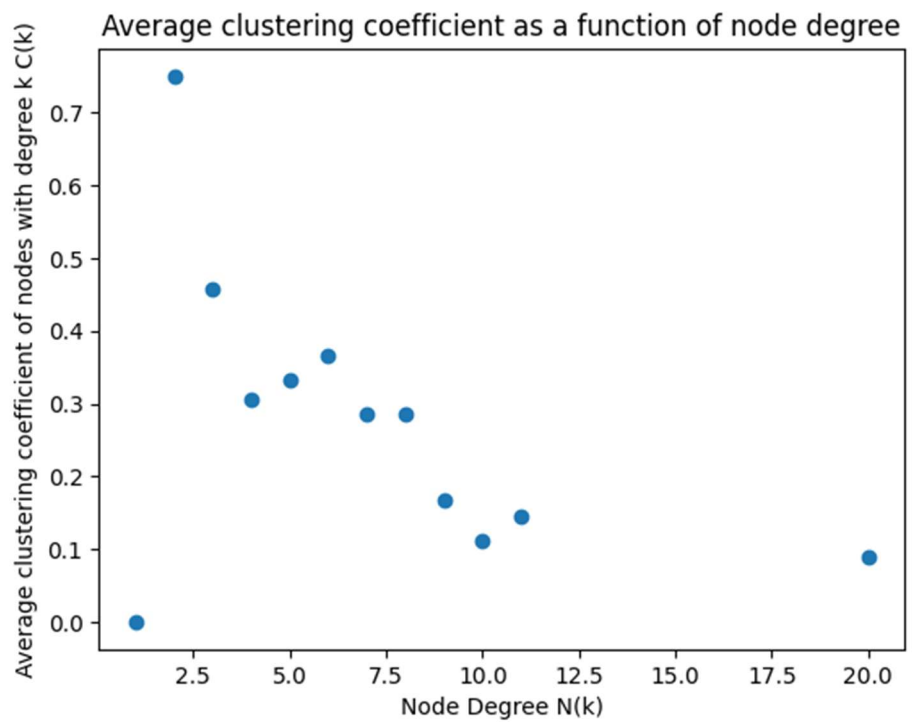


Figure 7: The average clustering coefficient versus node degree plot for the microbial association network in the Metro environment

- c. Upon running the Network Analyzer, Cytoscape provides the required network statistics in a tab within the Cytoscape environment. They are given as follows:

Environment	Average Degree	Average Clustering Coefficient	Characteristic Path Length
1. Hospital	12.241	0.392	2.147
2. Metro	4.512	0.297	2.555

Table 11: Network Parameters for different environments

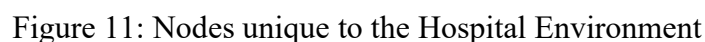
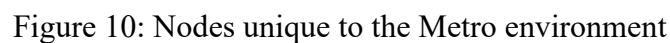
- The average degree for the Hospital environment is substantially higher than that for the Metro environment, indicating that an average species in the Hospital environment interacts with more species than an average species in the Metro environment.
- However, the average clustering coefficient and characteristic path length are comparable in both cases; while the Hospital has the higher average clustering coefficient, the Metro has the higher characteristic path length.
- Thus, the probability that two species that interact with a given species also interact with one another is comparable in both cases, based on the average clustering

d.

-
- Advanced Network Merge
- ☐ Union
 ☐ Intersection
 ☒ Difference
- ☐ Only remove nodes if all their edges are being subtracted, too
☒ Remove all nodes that are in the 2nd network
- Available Networks:
- Merged Network
- Networks to Merge:
- Metro
Hospital
- > <
- > Advanced Options
- Merge Cancel

[illegible]

Figure 9: The Intersection Network (nodes common to both the environments)



- To study how the normalized correlation values differ between the two environments, the interactions from the Edge Table in the Interaction Network were imported into a .csv file. These were maintained as separate columns by labelling them separately.
- An additional column was added to represent the differences in the edge weights, using the Function Builder option in Cytoscape, which would indicate how the normalized correlation values change from one environment to the other.
- These were used to obtain the following bar plot using the Matplotlib library in Python:

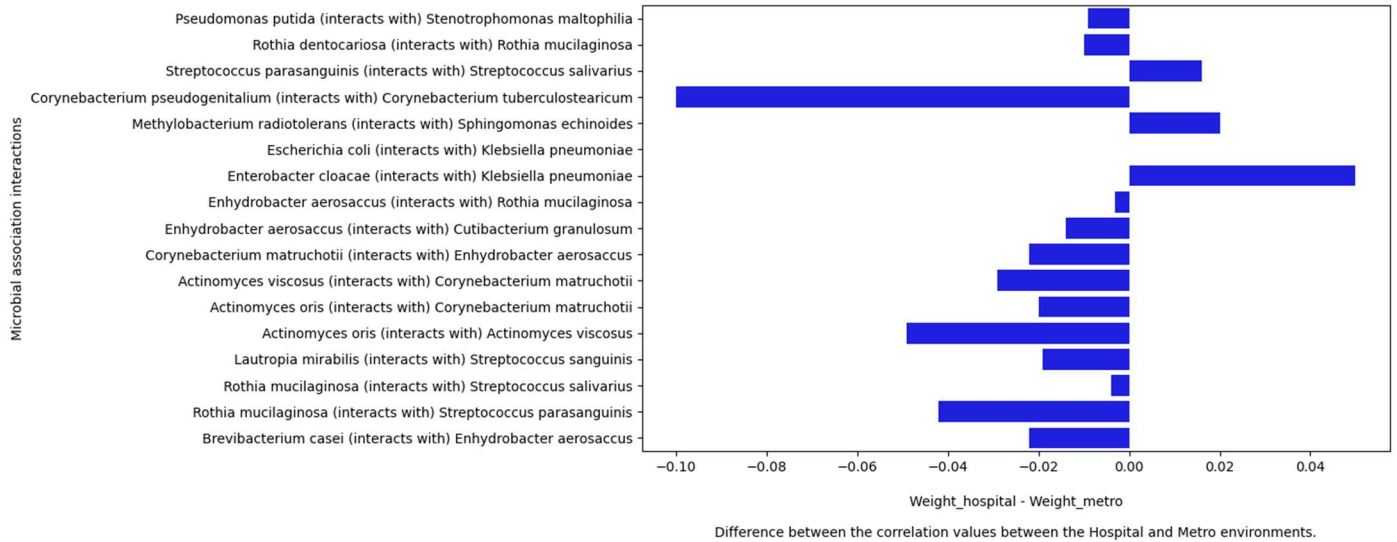


Figure 12: The differences between common microbial interactions in Hospital and Metro environments

- Thus, while the correlation values for most interactions are higher in the Metro environment, there are only three interactions that have higher correlation values in the Hospital environment.
- The top five interacting microbes in various contexts are given as follows:

Staphylococcus aureus	28
Micrococcus luteus	26
Klebsiella pneumoniae	25
Bifidobacterium longum	25
Staphylococcus capitis	25

Figure 13: In the original Hospital Network

Chroococcidiopsis thermalis	20
Caulobacter vibrioides	11
Rothia mucilaginosa	10
Kocuria sp. UCD-OTCP	9
Enhydrobacter aerosaccus	9

Figure 14: In the original Metro Network

Staphylococcus aureus	28
Micrococcus luteus	26
Staphylococcus capitis	25
Bifidobacterium longum	25
Veillonella parvula	24

Figure 15: Unique to the Hospital environment

Chroococcidiopsis thermalis	20
Caulobacter vibrioides	11
Kocuria sp. UCD-OTCP	9
Acinetobacter lwoffii	8
Deinococcus wulumuqiensis	7

Figure 16: Unique to the Metro environment

- We observe that the top two nodes with the maximum degree, in both cases, are unique to the network (for example, *Staphylococcus aureus* in the Hospital Network and *Chroococcidiopsis thermalis* in the Metro Network).
- Thus, the species that interact with a relatively high number of species tend to be unique to the environment.
- We also observe that the degrees of the nodes with the highest degree in the Hospital Network are higher than those in the Metro Network, indicating that species that have

more interactions in the Hospital environment tend to have more interactions than the most interacting species in the Metro environment.

- In both cases, we see examples of species that are common across the two environments, but are among the most interacting ones in one of the two environments (for example, *Klebsiella pneumoniae* in the Hospital Network and *Rothia mucilaginosa* in the Metro Network).

4. Acknowledgements:

- I discussed the arguments to be presented for the suitability of the z-test for part (e) of Question 1 with Varunkumar (BE23B016).
 - I discussed the arguments to be presented for the sub-question 5 about the resolution limit of the greedy modularity optimization algorithm of Question 2 with Oviya (BE23B010).
 - Sidharth (BS23B008) helped me locate the 'Merge' function required to observe the common and unique networks across different networks in the Tools tab in Cytoscape
 - I affirm that I have not copied any of the discussions, written content, or code verbatim from any source for this assignment.
-